FemboyScript

FemboyScript is the engine's deterministic gameplay and state-machine layer.

It has two authoring surfaces and two execution backends:

- visual LogicGraph and StateTree editors in the editor;
- a compact register bytecode executed by the editor VM;
- a cook-time C++26 AOT backend that produces a native plugin for builds.

This document describes the 0.3.0-alpha.1 behavior. FemboyScript 2.0 is the bytecode and graph format; the engine version is 0.3.0-alpha.1.

What exists in 0.3.0-alpha.1

Layer	Status
LogicGraph visual scripting	Editor panel and in-memory graph
StateTree visual scripting	Editor canvas plus TOML persistence in the scene
Bytecode compiler	`CompileLogicGraph()` produces canonical bytecode
Editor VM	`ScriptVm::Run()` executes bytecode without allocation
AOT transpiler	Bytecode to C++26, then a native plugin
Runtime AOT loader	`AotModule` validates ABI and component hashes
Standalone textual `.fbs` parser	`CompileTextScript()` text subset
Arrays, strings, maps, user functions, coroutines	Not present in 0.3.0-alpha.1

The visual graph is the default authoring surface for designers. The textual front-end is an optional power surface for version control, CI, mods and dense logic; it is part of the engine (`Engine/Script/TextScript.hpp`) and the editor's `VisualScriptPanel` text mode. It lowers the text subset to the same canonical bytecode as LogicGraph; the VM and AOT do not have a separate runtime path.

```
event update {
    let half = 0.5;
    self.Transform2D.Rotation = self.Transform2D.Rotation + half;
    if (self.Transform2D.Rotation > 3.14) {
        self.Transform2D.Rotation = 0.0;
    }
    log_float(self.Transform2D.Rotation);
}
```

Supported in the text subset: `start`/`update`/`fixed_update` blocks, `let`, assignment, `self.Component.Field`, `if`/`else`, `+ - * /`, comparisons, numeric literals, and native calls (`log_float`, `log_vec2`, `random_range`, ...).

Loops, arrays, strings and user functions are the next language version.

Execution model

An entity carries `ScriptComponent` with a program handle and event flags. The `FemboyScriptSystem` reads the handle, runs the `Start` event once and then `Update` every frame. `FixedUpdate` is available through the same entry table.

The VM is a register machine:

- 128 untyped 4-byte registers;
- 8-byte instructions;
- at most 65,536 instructions and 1,024 constants per program;
- at most 32 component bindings and 256 field bindings;
- 16 event functions and 32 native references;
- a 16,384-step budget per `Run()` call;
- no heap allocation on the run path;
- every fault is returned as a `VmFault`, never thrown.

Registers hold float, int32, uint32 or entity/handle bits. Type interpretation is the opcode's responsibility. Component access resolves a layout hash once at bind time, then performs a sparse-set lookup and a field-offset add.

Pipeline

```
LogicGraph or StateTree
        |
        v
CompileLogicGraph()
        |
        v
canonical FemboyScript bytecode (.fembak payload)
        |
        +--> ParseProgram() -> BindProgram() -> ScriptVm::Run()  editor/runtime
        |
        +--> TranspileProgramToCpp() -> C++26 -> native plugin       build
```

`ParseProgram()` is the trust boundary. It checks magic, version, table sizes, every register index, every jump target, every component/field binding, native argument ranges and opcode/field shape. After it returns true, the interpreter does not re-validate the hot path.

`BindProgram()` resolves component layout hashes through `EcsBridge`. A program

built against a different `LayoutHash` is rejected before the first field write.

LogicGraph visual scripting

`VisualScriptPanel` edits a `LogicGraph`:

- `Event Start/Update/FixedUpdate` entry nodes;
- math nodes (float, int, Vec2);
- field get/set nodes for Mirror components;
- branch and comparison nodes;
- native call nodes (`LogFloat`, `LogInt`, `LogVec2`, `RandomRange`, `SpawnPrefab`,

`DespawnSelf`, `PlaySound`, `GetDeltaTime`, `GetFrameIndex`);

- typed links and cycle rejection.

The graph is a fixed-capacity, trivially copyable payload and fits one

CommandStack external buffer. The editor undo/redo system operates on the graph

as data, not on C++ objects.

The graph is compiled with `CompileLogicGraph()`. The compiler rejects:

- missing event entry;
- unconnected required pins;
- type mismatches;
- cycles;
- more field/component bindings than the bytecode tables allow;
- graphs that exceed the instruction or register capacity.

StateTree visual scripting

`AIStateTreeCanvasPanel` edits a `StateTreeDefinition`. The runtime model is flat

and change-driven, not a behaviour-tree walk:

- fixed node table, maximum 64 nodes and 32 transitions;
- maximum depth 4;
- entity stores an active path (`ActiveStates[4]`) and a blackboard;
- evaluators are data (`TagsAny`, `TargetWithinRadius`, `FloatBelow`,

`StateTimeAtLeast`, ...);

- tasks emit `AICommandComponent` intent through the command buffer;
- a steady-state entity performs one evaluator and one task call.

The panel writes the compiled tree into the scene TOML:

```
[ai_state_tree]
version = 1
tree_id = 1
root = 0
node_count = 3
transition_count = 1

[[ai_state_tree.node]]
index = 0
kind = 0
parent = 65535
first_child = 1
next_sibling = 65535
child_count = 1
depth = 1
enabled = true
evaluator_kind = 0
evaluator_tag_mask = 0
evaluator_threshold = 0.0
evaluator_float_index = 0
task_kind = 0
task_tag_mask = 0
task_speed = 128.0
task_range = 32.0
task_flags = 0
position = [0.0, 0.0]

[[ai_state_tree.transition]]
from = 2
to = 1
triggers = 1
consume = 1
require_tags = 0
forbid_tags = 0
enabled = true
consume_all = false
```

`kind`: `0` Root, `1` Selector, `2` Leaf. `parent`, `first_child`, `next_sibling` are indices; `65535` means none. `evaluator_kind` and `task_kind` use the enum order in `StateTree.hpp`. `position` is editor-only canvas data.

The editor button `Save AI TOML` rewrites only the `[ai_state_tree]` block and preserves the rest of the scene. `Load AI TOML` validates the block and rebuilds the canvas. Hand editing is supported, but node indices and linked-list fields must remain consistent.

Bytecode format

The header is one 64-byte cache line:

```
magic u32, version u16, flags u16,
codeCount u32, constantCount u32, fieldCount u32, componentCount u32,
functionCount u32, nativeCount u32, registerHighWater u32, reserved u32,
sourceHash u64, graphHash u64, reserved2 u64
```

Tables follow the header in this order: code, constants, fields, components, functions, native ids. Every table starts at a fixed alignment; the total size is aligned to `max_align_t`. The format is canonical and deterministic: the same graph produces byte-identical bytes on every machine.

An instruction is `op:u8 dst:u8 src:u8 flags:u8 operand:u16 reserved:u16`.

Opcodes are closed: `Nop`, `Halt`, load/move, float/int arithmetic, bitwise int ops, comparisons, jumps, entity/field access, native call, spawn/despawn and random. A new feature is a native function, not an ad-hoc opcode.

VM fault codes

`InvalidProgram`, `MissingComponent`, `BadEntity`, `BadNative`, `BadField`, `BadRegister`, `StepLimit`, `DivisionByZero`, `NoRandomSource`, `NoCommandBuffer`, `SpawnUnavailable`, `UnknownOpcode`.

`ScriptRuntime::ReportFault()` logs at most once per program slot. `Start` is marked started even after a fault, so a broken Start does not flood the log at 60 Hz.

AOT backend

`AotBuilder` runs on a worker:

1. `TranspileLogicGraphToCpp()` compiles the graph to bytecode, validates it and emits a flat C++26 function per event;
2. generated code includes only `Engine/Script/AotAbi.hpp`;
3. component access goes through `Aot::Context::Component(LayoutHash)`;
4. the plugin exports `fby_aot_module()`, a `Module` record with component hashes and `start/update/fixedUpdate` pointers;

5. `AotModule::Load()` validates magic, ABI version, component hashes and the presence of at least one event before executing anything.

The generated code has no VM dispatch loop and no bytecode reads. It still uses the same component LayoutHash contract as the editor. A stale plugin is rejected instead of writing into a changed component.

Arithmetic parity is enforced: integer add/sub/mul/neg use defined unsigned wrap; `INT_MIN / -1` and `INT_MIN % -1` are guarded; non-finite float constants are emitted as `std::numeric_limits` expressions.

AOT parity is closed in the current ABI (version 7): the host log callback is signed (`logI32`), `DespawnSelf` returns the same 1/0 result the VM writes, physics and camera callbacks are shared with the VM, and generated code calls a `fault(code, pc)` hook, with fault telemetry kept in `AotModule` instead of failing silently.

Editor workflow

6. Open `VisualScript` and build a LogicGraph, or open `AI State Tree` and build a state tree.
7. Compile the graph and inspect the reported node/instruction/register counts.
8. Save the StateTree block to the scene TOML; the LogicGraph is an editor asset payload in 0.3.0-alpha.1.
9. Assign a `ScriptComponent` program handle to the entity.
10. Run PIE; faults appear in the Console/LogBoy panel with program, entity and pc.
11. For a build, enable AOT and run `fby_aotc` through the release pipeline. The resulting plugin is validated by `AotModule`.

Physics backend API

The default backend is `Auto`. The three natives are available in every script and compile to the same behaviour in VM and AOT:

```
backend = physics_get_backend()       // 1 Box2D, 2 DOD, -1 unavailable
caps = physics_get_capabilities()     // bit 0 joints, 1 CCD, 2 soft, 3 deterministic,
4 large
ok = physics_set_backend(backend)     // updates the project policy for the next world
```

`physics_set_backend` does not swap a live solver. It updates the policy copied into the next Play-In-Editor or game launch; the current world keeps running so a mid-frame change can never invalidate body handles. Query capabilities before authoring a graph that depends on joints or CCD.

The LogicGraph palette exposes the same operations as `Physics Get Backend`, `Physics Set Backend` and `Physics Capabilities`. Their output pin is Float; the Set node takes Exec and a Float backend selector. Existing graphs without these nodes migrate as `Auto`.

Projectiles are tuned through the reflected `Projectile2D` component rather than a separate native: `Mode` (0 Auto, 1 Hitscan, 2 Swept, 3 Dynamic), `Speed`, `Radius`, `MinWallThickness`, `Direction`, `MaxSubsteps` and `CollisionMask` can be set with the regular component field nodes.

Camera motion and distortion

The camera API operates on the active `CameraStackSystem` and never exposes raw camera handles. Effects are cosmetic and are not part of rollback snapshots.

```
camera_add_trauma(amount)
camera_distort(flags, strength, paramX, paramY, duration)
camera_offset(x, y, duration)
camera_zoom(zoom, duration)
camera_roll(radians, duration)
camera_clear_effects()
camera_set_motion_scale(scale)
```

`camera_distort` flags: `1` double vision, `2` anisotropic stretch (`paramX/Y` are the X/Y scale), `4` radial warp, `8` chromatic split, `16` wobble (`strength` is amplitude, `paramX` is frequency). A game defines an intoxicated or poisoned state itself and maps it to these generic primitives; the engine does not label the gameplay state.

The visual palette exposes `Camera Add Trauma`, `Camera Distort`, `Camera

Offset, Camera Zoom, Camera Roll, Camera Clear Effects and Camera Motion Scale`. They compile to the same natives, and AOT plugins call the versioned camera callbacks.

Limits and failure behavior

- Loading more than 32 programs fails with `script program table is full`.

Call `UnloadProgram()` on hot reload.

- A program whose registers exceed 128 is rejected at load.
- A `CallNative` whose arguments exceed the register file is rejected at load.
- A field binding whose size does not match the opcode is rejected at load.
- A missing component faults at runtime before any write.
- A missing command buffer faults `DespawnSelf`/`SpawnPrefab` paths.

Debugging

A debug build carries an optional debug section after the program tables: one `DebugSpan { pc, line, column, flags }` per instruction, named register locals and the source name. The section is flagged in `ProgramHeader::flags`, so a program without it loads unchanged and old readers ignore the trailing bytes.

`TextScript` emits a span per instruction with 1-based line/column and a `DebugVariable` per `let`; the LogicGraph compiler emits one graph-node span per emitted node range (`DebugSpanGraphNode`).

`ScriptVm::RunDebug()` executes the same interpreter with a caller-owned `VmDebugState`: breakpoints by pc (`SetBreakpoint`, 8 KiB bitmap), single-step budget, pause request, and a resume that keeps the register file and pc intact. `VmStopInfo` reports `Completed`, `Faulted` or `Paused` plus the reason and the pc. `ScriptDebug` helpers map a pc back to a source location, list the program's fields (resolved against the scene schema), and read/write live field values.

The editor's **Script Debugger** panel compiles the text script, shows the source with clickable breakpoints, Continue/Step/Pause/Reset, a Locals table, editable

Self fields, a one-frame stack and program counters. `ScriptDebuggerPanelTest` covers compile errors, a line breakpoint, resume and fault reporting without an ImGui frame.

Roadmap

The next language features, in dependency order:

12. Arrays, strings, maps and user functions with a fixed call stack.
13. Coroutines/timers and deterministic scheduling.
14. Debugger views for the StateTree canvas (the VM-side breakpoints, spans and the LogicGraph "Debug graph" view already exist).

The flat VM contract is not extended by adding heap objects to the 8-byte instruction. Any new object model needs a new bytecode version and a migration path.